JAILEDDOTFUN ENHANCEMENT 3
Security Review



MARCH, 2026

www.chaindefenders.xyz
https://x.com/DefendersAudits

Lead Auditors



PeterSR



0x539.eth

Table of Contents

1. Protocol Summary
2. Disclaimer
3. Risk Classification
4. Audit Details

- Commit Hash
- Scope
- Roles

5. Executive Summary

- Issues Found
- Issues Statuses
- Security Grade

6. Findings

- Low
- Informational

Protocol Summary

A Solana-based pickaxe training and exploration game. It blends NFT-style assets, STEPN-like exploration loops, and a Token-2022 economy with JAIL. On-chain handles ownership and verification; off-chain handles computation and player experience.

This update introduces marketplace purchase rate limiting for Pickaxe and PickaxeBox items to prevent abuse and ensure fair access. It adds per-player `PlayerPurchaseRateLimit` PDA tracking along with new configuration fields in `MarketplaceConfig` to define and manage rate limit parameters. The rate limit is enforced during `buy_item` execution using `remaining_accounts` when applicable, ensuring purchases comply with configured restrictions.

Additionally, this update introduces a new admin instruction, `update_purchase_rate_limit`, along with the `PurchaseRateLimitUpdated` event, enabling administrators to modify rate limiting settings transparently. A new player instruction, `burn_owned_pickaxe_box`, allows users to clean up their inventory by burning owned PickaxeBoxes.

Finally, this update aligns inventory capacity validation by correcting the inventory-full error messaging to reflect the actual maximum limit of 128 items instead of the previously incorrect 256.

Disclaimer

The Chain Defenders team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation of the contracts.

Risk Classification

Impact/Likelihood	High	Medium	Low
High	High	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Audit Details

Commit Hash

2c2fe425237ae8c59df80b1eafe1058b9d5289cc

Scope

Id	Files in scope
1	./constants.rs
2	./errors.rs
3	./heap.rs
4	./lib.rs
5	./state.rs
6	./instructions/mod.rs
7	./instructions/admin/admin.rs
8	./instructions/admin/burn.rs
9	./instructions/admin/mod.rs
10	./instructions/boxes/mod.rs
11	./instructions/boxes/mystery_rewards.rs
12	./instructions/boxes/pickaxe_boxes.rs
13	./instructions/breeding/mod.rs
14	./instructions/breeding/pickaxe.rs
15	./instructions/breeding/vrf.rs
16	./instructions/equipment/gem_inventory.rs
17	./instructions/equipment/gem_level.rs
18	./instructions/equipment/gems.rs
19	./instructions/equipment/initialize.rs
20	./instructions/equipment/mod.rs
21	./instructions/equipment/pickaxe_inventory.rs
22	./instructions/equipment/pickaxe_repair.rs
23	./instructions/exploration/loadout.rs
24	./instructions/exploration/mod.rs
25	./instructions/exploration/rewards.rs

Id	Files in scope
26	./instructions/exploration/start.rs
27	./instructions/marketplace/marketplace.rs
28	./instructions/marketplace/mod.rs
29	./instructions/token/jail_token.rs
30	./instructions/token/mod.rs
31	./utils/domain.rs
32	./utils/mod.rs
33	./utils/shared.rs
34	./utils/signature.rs
35	./utils/vrf_helpers.rs

Note: The scope of the audit was PR 61.

Roles

Id	Roles
1	Admin
2	Operator
3	User

Executive Summary

Issues Found

Severity	Count	Description
High	0	Critical vulnerabilities
Medium	0	Significant risks
Low	1	Minor issues with low impact
Informational	2	Best practices or suggestions
Gas	0	Optimization opportunities

Issues Statuses

Id	Title	Status
L-01	Incomplete Account Closure In ‘close_player_accounts‘	Acknowledged
I-01	Manual Byte-Level Manipulation On The ‘PlayerPurchaseRateLimit‘ Account	Fixed
I-02	Missing New Fields In Event Emission	Fixed

Security Grade



Based on the audit findings and code quality, the overall protocol security grade is 99.00 /100.00. This reflects an excellent security posture with almost no room for improvement.

Findings

Low

Low 01 Incomplete Account Closure In `close_player_accounts`

Location

admin.rs

Description

The `close_player_accounts` instruction is intended to permanently close and clean up a player's on-chain state during game shutdown.

However, it only closes:

- `PlayerProfile`
- `PlayerPickaxeInventory`

But additional related accounts may exist:

- `PlayerRateLimit`
- Marketplace `Listings`
- `BreedingAttempt`
- `GemLevelUpAttempt`
- `PlayerAccessRecord`

These accounts are not closed by this instruction.

If any of these accounts remain open:

- Rent remains locked
- Orphaned state may persist
- Program invariants may break
- Future migrations or shutdown accounting may be incomplete
- Off-chain systems may assume full deletion while partial state remains

This creates inconsistent shutdown behavior and may leave residual state after a supposed full account purge.

Recommendation

Option A — Full Cleanup Enforcement (Recommended)

Extend `ClosePlayerAccounts` to:

- Require all related PDAs as optional accounts
- Close them via `close = admin`
- Emit destruction counts per account type

This ensures:

- Complete state deletion
- No rent leakage
- No orphaned state

Option B — Precondition Enforcement

Require that:

- No active listings
- No breeding attempts
- No gem level-up attempts
- No rate limit account

Add explicit `require!` checks before closure.

This ensures:

- State consistency before destruction
- No dangling references

Status

Acknowledged

Informational

Info 01 Manual Byte-Level Manipulation On The `PlayerPurchaseRateLimit` Account

Location

marketplace.rs

Description

The `enforce_purchase_rate_limit_via_remaining` function in `marketplace.rs` performs low-level byte-level manipulation on the `PlayerPurchaseRateLimit` account to read and write discriminator, bump, and timestamp fields. Specifically:

- Discriminator write:

`data[..8].copy_from_slice(PlayerPurchaseRateLimit::DISCRIMINATOR)` at initialization

- Bump write:

`data[88] = bump` assumes bump field is at offset 88

- Timestamp reads/writes:

Loop reads timestamps at offsets `8 + i*8` (for i = 0..9)

This approach hardcodes memory layout offsets and relies on exact struct size assumptions. If the `PlayerPurchaseRateLimit` struct definition changes (e.g., fields are added, removed, reordered, or `#[repr( ... )]` changes), these hardcoded offsets become incorrect. This can lead to:

1. Data corruption: Writing/reading from incorrect memory locations, corrupting account data
2. Panics: The `try_into().unwrap()` call at will panic if buffer slices are out of bounds
3. Logic errors: Timestamp validation could read stale or corrupted data, causing unexpected purchase rejections or allowing rate-limit bypasses
4. Maintenance risk: Future developers modifying the `PlayerPurchaseRateLimit` struct may not realize the tight coupling to these byte offsets, introducing subtle bugs

Recommendation

Replace manual byte-level operations with Anchor account deserialization and serialization APIs:

1. Preferred: Refactor to use `Account<'info, PlayerPurchaseRateLimit>` from the instruction accounts context, allowing Anchor to handle all serialization/deserialization automatically. This eliminates byte-offset coupling.
2. Alternative: If manual initialization is necessary, centralize all offset constants in a module and validate struct layout at compile time using macros to ensure `PlayerPurchaseRateLimit::LEN` matches the expected size. Document the exact field layout and mark it as a stability contract.
3. Example safer approach:

```
// In your code, instead of manual offsets:
let mut rate_limit = account_zero_init::<PlayerPurchaseRateLimit>( ... )?;
rate_limit.timestamps[oldest_idx] = now;
rate_limit.serialize(&mut *rate_limit_info.try_borrow_mut_data()?)?;
```

This ensures struct layout changes are caught by the type system rather than causing silent data corruption at runtime.

Status

Fixed

Info 02 Missing New Fields In Event Emission

Location

marketplace.rs

Description

In the `marketplace.rs::initialize_marketplace_config` function, three purchase rate limit configuration fields are initialized on the `MarketplaceConfig` account:

```
marketplace_config.purchase_rate_limit_enabled = true;
marketplace_config.purchase_rate_limit_max_count =
    crate::constants::purchase_rate_limit::DEFAULT_MAX_COUNT;
marketplace_config.purchase_rate_limit_window_secs =
    crate::constants::purchase_rate_limit::DEFAULT_WINDOW_SECS;
```

However, the emitted `MarketplaceConfigInitialized` event does not include these three fields:

```
emit!(MarketplaceConfigInitialized {
    authority: marketplace_config.authority,
    admin: ctx.accounts.admin.key(),
    treasury_shards: marketplace_config.treasury_shards,
    fee_rate,
});
```

This omission reduces observability during marketplace initialization. Off-chain indexers and monitoring systems cannot see the initial rate-limit configuration that was set, requiring them to make RPC calls or infer defaults. If the defaults are later changed in constants, historical records will be incomplete or misleading about what was actually initialized.

Additionally, when `update_purchase_rate_limit` is called later, a `PurchaseRateLimitUpdated` event is emitted with full details, creating an inconsistency: the initial configuration is not logged, but all subsequent updates are.

Recommendation

Add the three purchase rate limit fields to the `MarketplaceConfigInitialized` event struct and emit them:

```
#[event]
pub struct MarketplaceConfigInitialized {
    pub authority: Pubkey,
    pub admin: Pubkey,
    pub treasury_shards: [Pubkey; 4],
```

```
    pub fee_rate: u16,
    pub purchase_rate_limit_enabled: bool,
    pub purchase_rate_limit_max_count: u8,
    pub purchase_rate_limit_window_secs: u32,
}

// ... existing code ...

    emit!(MarketplaceConfigInitialized {
        authority: marketplace_config.authority,
        admin: ctx.accounts.admin.key(),
        treasury_shards: marketplace_config.treasury_shards,
        fee_rate,
        purchase_rate_limit_enabled: marketplace_config.purchase_rate_limit_enabled,
        purchase_rate_limit_max_count: marketplace_config.purchase_rate_limit_max_count,
        purchase_rate_limit_window_secs: marketplace_config.
    purchase_rate_limit_window_secs,
    });
```

This ensures complete initialization transparency and consistency with the `PurchaseRateLimitUpdated` event pattern.

Status

Fixed